DOCUMENTAÇÃO DO PROJETO

Quiz Muito Louco!

Manual Técnico

Arquivos em ordem de execução, o que cada função faz e o papel de cada variável e recurso do PHP usado no código.

Tecnologias	**PHP 8.2 procedural · JSON · HTML · CSS · JavaScript**
Servidor	**Apache (XAMPP)**
Estilo de código	**Object Calisthenics adaptado ao procedural**

Sumário

1. Visão geral
2. Dados e proteção (`data/`, `.htaccess`)
3. Funções base (`includes/`)
4. Templates (`templates/`)
5. Páginas, na ordem do jogador
6. Estilos e scripts
7. Glossário de funções e recursos do PHP

1. Visão geral

O jogo é um quiz de 10 perguntas sorteadas. O jogador cria uma conta, responde às perguntas contra o relógio e entra no ranking. O código é **PHP procedural** (só funções, sem classes) e os dados ficam em **arquivos JSON**, sem banco de dados.

Estrutura de pastas

```
Quiz/
├── .htaccess              bloqueia data/, includes/ e templates/ no navegador
├── index.php              tela inicial (login e cadastro)
├── authenticate.php       processa login/cadastro (sem HTML)
├── logout.php             encerra a sessão (sem HTML)
├── pages/
│   ├── game/
│   │   ├── game.php       tela da pergunta
│   │   ├── answer.php     processa a resposta (sem HTML)
│   │   ├── result.php     resultado da partida
│   │   ├── css/game.css
│   │   └── js/game.js
│   └── ranking/
│       ├── ranking.php    ranking
│       └── css/ranking.css
├── includes/              só funções, nenhum HTML
├── templates/             header.php e footer.php
├── data/                  questions.json e users.json
└── assets/                global.css e script.js
```

Fluxo das páginas

`index.php` POST → `authenticate.php` ok → `game.php` POST → `answer.php` fim → `result.php` → `ranking.php`

Enquanto houver perguntas, `answer.php` volta para `game.php`. Em caso de erro no login, `authenticate.php` volta para `index.php` com uma mensagem.

Padrão Post/Redirect/Get

Arquivos que recebem formulário (`authenticate.php`, `answer.php`) **não imprimem HTML**: processam e redirecionam. Assim, apertar F5 nunca reenvia uma resposta.

Convenções

ONDE	PADRÃO	EXEMPLO
Chaves de JSON e da sessão	inglês, `snake_case`	`best_time_seconds`
Funções e variáveis	inglês, `camelCase`	`findUserByName()`, `$currentQuestion`

Constantes	`UPPER_SNAKE_CASE`	`QUESTIONS_PER_GAME`
Textos na tela	português	"Nome ou senha inválidos."

Object Calisthenics aplicado

REGRA	COMO APARECE NO CÓDIGO
1. Um nível de indentação	Nenhuma função tem `if` dentro de `foreach`; usamos `array_filter`/`array_map`.
2. Sem `else`	`return` antecipado (guard clauses) e `redirect()`, que encerra o script.
3. Encapsular primitivos	Regras com nome: `isValidName()`, `isValidOption()`.
4. Coleções de primeira classe	Cada lista tem seu arquivo: usuários em `users.php`, perguntas em `questions.php`.
5. Um ponto por linha	Nada de `$_SESSION['game']['question_ids'][...]`; usamos `gameValue()` e `currentQuestionId()`.
6. Não abreviar	`$question`, nunca `$q`.
7. Entidades pequenas	Funções de até ~10 linhas; arquivos de até ~60 linhas.
8. Poucos parâmetros	No máximo 3 parâmetros por função.
9. Tell, don't ask	`saveBestGame()` decide sozinha se grava o recorde.

2. Dados e proteção

`data/questions.json`

Lista com as 45 perguntas. Exemplo de um item:

```
{
    "id": 1,
    "question": "Qual é a capital do Brasil?",
    "options": ["São Paulo", "Rio de Janeiro", "Brasília", "Salvador"],
    "answer": 2
}
```

CHAVE	SIGNIFICADO
`id`	Identificador único da pergunta.
`question`	Enunciado exibido ao jogador.
`options`	As 4 alternativas. A posição na lista (0 a 3) é o "índice" da opção.
`answer`	Índice da opção correta. No exemplo, `2` = "Brasília".

`data/users.json`

```
{
    "id": 1,
    "name": "Maria",
    "password": "maria123",
    "best_correct_answers": 8,
    "best_time_seconds": 25.6
}
```

CHAVE	SIGNIFICADO
`id`	Identificador único do usuário.
`name`	Nome usado no login (único, sem diferenciar maiúsculas).
`password`	Senha em texto puro (escolha consciente do projeto acadêmico).
`best_correct_answers`	Acertos da melhor partida.
`best_time_seconds`	Tempo da melhor partida. `null` = nunca jogou.

`.htaccess`

```
RewriteEngine On
RewriteRule ^(data|includes|templates)/ - [F]
```

COMANDO	O QUE FAZ
`RewriteEngine On`	Liga o *mod_rewrite*, módulo do Apache que analisa a URL pedida. Sem ele, a regra seguinte é ignorada.
`RewriteRule`	Define uma regra: *padrão*, *destino* e *flags*.
`^(data|includes|templates)/`	Expressão regular: URL que **começa** (`^`) com `data/`, `includes/` **ou** (`|`) `templates/`.
`-`	Não reescreve a URL.
`[F]`	*Forbidden*: responde 403 e não entrega o arquivo.

> O bloqueio vale só para o navegador (HTTP). O PHP lê esses arquivos direto do disco, então o jogo continua funcionando.

3. Funções base — includes/

Os arquivos a seguir aparecem na ordem em que o `boot.php` os carrega. Nenhum deles imprime HTML.

`includes/boot.php`

Primeiro arquivo carregado por **toda** página. Prepara o sistema antes de qualquer outra coisa rodar.

```
<?php

declare(strict_types=1);

session_start();

const ROOT_PATH = __DIR__ . '/..';
const BASE_URL = '/Quiz/';
const USERS_FILE = ROOT_PATH . '/data/users.json';
const QUESTIONS_FILE = ROOT_PATH . '/data/questions.json';

const QUESTIONS_PER_GAME = 10;
const NAME_MIN_LENGTH = 3;
const NAME_MAX_LENGTH = 30;
const PASSWORD_MIN_LENGTH = 6;

require_once __DIR__ . '/json.php';
require_once __DIR__ . '/collection.php';
require_once __DIR__ . '/view.php';
require_once __DIR__ . '/session.php';
require_once __DIR__ . '/users.php';
require_once __DIR__ . '/auth.php';
require_once __DIR__ . '/ranking.php';
require_once __DIR__ . '/questions.php';
require_once __DIR__ . '/game.php';
require_once __DIR__ . '/timer.php';
require_once __DIR__ . '/result.php';
```

ITEM	O QUE FAZ
`declare(strict_types=1)`	Ativa a tipagem estrita: passar texto onde se espera `int` gera erro, em vez de converter sozinho.
`session_start()`	Inicia (ou retoma) a sessão do visitante, liberando o uso de `$_SESSION`.
`ROOT_PATH`	Caminho da raiz do projeto no disco (`__DIR__` = pasta do arquivo atual; `/..` sobe um nível).
`BASE_URL`	Início de todas as URLs do site: `/Quiz/`.
`USERS_FILE`, `QUESTIONS_FILE`	Caminhos dos dois arquivos JSON.
`QUESTIONS_PER_GAME`	Quantidade de perguntas por partida (10).
`NAME_MIN_LENGTH`, `NAME_MAX_LENGTH`	Tamanho mínimo (3) e máximo (30) do nome.

`PASSWORD_MIN_LENGTH`	Tamanho mínimo da senha (6).
`require_once`	Carrega cada arquivo de funções uma única vez.

includes/json.php — leitura e escrita dos arquivos

Únicas funções do projeto que tocam nos arquivos JSON.

```
<?php

declare(strict_types=1);

function readJson(string $path): array
{
    $content = file_get_contents($path);

    if ($content === false || trim($content) === '') {
        return [];
    }

    return json_decode($content, true) ?? [];
}

function saveJson(string $path, array $data): void
{
    $json = json_encode($data, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES);

    file_put_contents($path, $json . "\n", LOCK_EX);
}
```

FUNÇÃO	VARIÁVEIS E RECURSOS	O QUE FAZ
`readJson($path)`	`$path`: caminho do arquivo. `$content`: texto lido, ou `false` se falhar. `file_get_contents`, `trim`, `json_decode(..., true)`, `??`	Lê o arquivo inteiro. Se não conseguiu ler ou está vazio, devolve `[]`. Senão converte o texto JSON em array (`true` = array associativo). Se o JSON for inválido, `?? []` devolve lista vazia.
`saveJson($path, $data)`	`$data`: array a gravar. `$json`: texto JSON gerado. `json_encode`, `file_put_contents`, `LOCK_EX`	Converte o array em JSON e grava no arquivo. Flags: `JSON_PRETTY_PRINT` (indentado), `JSON_UNESCAPED_UNICODE` (acentos legíveis), `JSON_UNESCAPED_SLASHES` (não escapa `/`). `LOCK_EX` trava o arquivo durante a gravação: se dois jogadores salvarem ao mesmo tempo, um espera o outro.

includes/collection.php — busca em listas

```
<?php
```

```php
declare(strict_types=1);

function findFirst(array $items, callable $condition): ?array
{
    $matches = array_values(array_filter($items, $condition));

    return $matches[0] ?? null;
}
```

FUNÇÃO	VARIÁVEIS E RECURSOS	O QUE FAZ
`findFirst($items, $condition)`	`$items`: lista onde buscar. `$condition`: `callable`, uma função que recebe um item e devolve `true`/`false` (um *callback*, como no JavaScript). `$matches`: itens aprovados. `array_filter`, `array_values`	Filtra a lista com a condição, reorganiza os índices a partir de 0 (`array_values`) e devolve o primeiro item. Se nenhum passou, `$matches[0]` não existe e `?? null` devolve `null`. Usada para buscar usuários e perguntas.

includes/view.php — apoio às telas

```php
<?php

declare(strict_types=1);

function escape(string $text): string
{
    return htmlspecialchars($text, ENT_QUOTES, 'UTF-8');
}

function url(string $path): string
{
    return BASE_URL . $path;
}

function redirect(string $path): never
{
    header('Location: ' . url($path));
    exit;
}

function isPostRequest(): bool
{
    return $_SERVER['REQUEST_METHOD'] === 'POST';
}

function formatTime(?float $seconds): string
{
    if ($seconds === null) {
        return '-';
    }

    $minutes = intdiv((int) $seconds, 60);
    $remainingSeconds = number_format($seconds - $minutes * 60, 1, ',', '');

    if ($minutes === 0) {
        return $remainingSeconds . 's';
```

```
    }

    return $minutes . 'min ' . $remainingSeconds . 's';
}
```

FUNÇÃO	VARIÁVEIS E RECURSOS	O QUE FAZ
`escape($text)`	`htmlspecialchars` com `ENT_QUOTES` e `UTF-8`	Troca `<`, `>`, `&` e aspas por códigos HTML. Um nome como `<script>` aparece como texto, sem ser executado.
`url($path)`	`$path`: caminho a partir da raiz. Operador `.` (concatenação)	`'pages/game/game.php'` vira `'/Quiz/pages/game/game.php'`.
`redirect($path)`	`header('Location: ...')`, `exit`, retorno `never`	Manda o navegador para outra página e encerra o script. O tipo `never` indica que a função nunca retorna.
`isPostRequest()`	`$_SERVER['REQUEST_METHOD']`	Diz se a requisição é um envio de formulário (POST).
`formatTime($seconds)`	`$seconds`: `?float` (número ou `null`). `$minutes`: minutos inteiros (`intdiv`). `$remainingSeconds`: segundos restantes com 1 casa e vírgula (`number_format`).	`83.4` → `"1min 23,4s"`; `4.25` → `"4,3s"`; `null` → `"-"`.

includes/session.php — sessão e mensagens

```
<?php

declare(strict_types=1);

function startUserSession(array $user): void
{
    session_regenerate_id(true);
    $_SESSION['user_id'] = $user['id'];
}

function endUserSession(): void
{
    $_SESSION = [];
    session_destroy();
}

function loggedUserId(): ?int
{
    return $_SESSION['user_id'] ?? null;
}

function requireLogin(): void
{
    if (loggedUserId() === null) {
        redirect('index.php');
    }
```

```
}

function setFlash(string $message, string $type = 'success'): void
{
    $_SESSION['flash'] = ['message' => $message, 'type' => $type];
}

function pullFlash(): ?array
{
    $flash = $_SESSION['flash'] ?? null;
    unset($_SESSION['flash']);

    return $flash;
}
```

FUNÇÃO	VARIÁVEIS E RECURSOS	O QUE FAZ
`startUserSession($user)`	`$user`: usuário que fez login. `session_regenerate_id(true)`, `$_SESSION['user_id']`	Gera um novo id de sessão (evita que alguém reaproveite um id antigo) e guarda quem está logado.
`endUserSession()`	`$_SESSION = []`, `session_destroy()`	Apaga todos os dados da sessão e a destrói.
`loggedUserId()`	`?? null`	Devolve o id de quem está logado, ou `null`.
`requireLogin()`	chama `redirect()`	Barreira das páginas internas: sem login, volta para `index.php`.
`setFlash($message, $type)`	`$message`: texto. `$type`: `'success'` (padrão) ou `'error'`. `$_SESSION['flash']`	Guarda uma mensagem para mostrar na próxima tela.
`pullFlash()`	`$flash`: mensagem guardada. `unset`	Lê a mensagem e a apaga da sessão, para aparecer uma única vez.

includes/users.php — usuários

```
<?php

declare(strict_types=1);

function listUsers(): array
{
    return readJson(USERS_FILE);
}

function saveUsers(array $users): void
{
    saveJson(USERS_FILE, array_values($users));
}

function findUserByName(string $name): ?array
{
```

```
    $normalizedName = mb_strtolower($name);

    return findFirst(listUsers(), fn (array $user) => mb_strtolower($user['name']) === $normalizedName);
}

function findUserById(int $id): ?array
{
    return findFirst(listUsers(), fn (array $user) => $user['id'] === $id);
}

function nextUserId(array $users): int
{
    return max([0, ...array_column($users, 'id')]) + 1;
}

function isValidName(string $name): bool
{
    $length = mb_strlen($name);

    return $length >= NAME_MIN_LENGTH && $length <= NAME_MAX_LENGTH;
}

function isValidPassword(string $password): bool
{
    return mb_strlen($password) >= PASSWORD_MIN_LENGTH;
}

function registerUser(string $name, string $password): array
{
    $users = listUsers();
    $user = [
        'id' => nextUserId($users),
        'name' => $name,
        'password' => $password,
        'best_correct_answers' => 0,
        'best_time_seconds' => null,
    ];

    saveUsers([...$users, $user]);

    return $user;
}

function replaceUser(array $updatedUser): void
{
    $users = array_map(
        fn (array $user) => $user['id'] === $updatedUser['id'] ? $updatedUser : $user,
        listUsers()
    );

    saveUsers($users);
}
```

FUNÇÃO	VARIÁVEIS E RECURSOS	O QUE FAZ
`listUsers()`	`readJson(USERS_FILE)`	Lê todos os usuários.
`saveUsers($users)`	`array_values`	Grava a lista; `array_values` garante índices 0, 1, 2... para o JSON ser uma lista `[...]`.

`findUserByName($name)`	`$normalizedName`: nome em minúsculas (`mb_strtolower`, que funciona com acentos). `fn (array $user) => ...`: *arrow function* usada como condição.	Busca o usuário comparando os nomes em minúsculas: "maria" encontra "Maria".
`findUserById($id)`	arrow function com `===`	Busca o usuário pelo id.
`nextUserId($users)`	`array_column` (pega só os ids), `...` (espalha a lista), `max`	Maior id existente + 1. O `0` na lista garante que, sem usuários, o primeiro id seja 1.
`isValidName($name)`	`$length`: `mb_strlen` (conta letras acentuadas como 1)	Nome entre 3 e 30 caracteres.
`isValidPassword($password)`	`mb_strlen`	Senha com pelo menos 6 caracteres.
`registerUser($name, $password)`	`$users`: lista atual. `$user`: novo usuário, sem partidas. `[...$users, $user]`: lista antiga + o novo.	Cria o usuário, grava e o devolve.
`replaceUser($updatedUser)`	`$users`: lista nova. `array_map` + operador ternário `? :`	Percorre todos os usuários; o que tiver o mesmo id é trocado pela versão atualizada. Depois grava.

includes/auth.php — login e cadastro

```php
<?php

declare(strict_types=1);

function authenticate(string $name, string $password): ?array
{
    $user = findUserByName($name);

    if ($user === null) {
        return null;
    }

    if ($user['password'] !== $password) {
        return null;
    }

    return $user;
}

function registrationError(string $name, string $password): ?string
{
    if (!isValidName($name)) {
        return 'O nome deve ter entre ' . NAME_MIN_LENGTH . ' e ' . NAME_MAX_LENGTH . ' caracteres.';
    }

    if (!isValidPassword($password)) {
        return 'A senha deve ter pelo menos ' . PASSWORD_MIN_LENGTH . ' caracteres.';
    }
```

```
    if (findUserByName($name) !== null) {
        return 'Esse nome já está em uso.';
    }

    return null;
}

function failAuthentication(string $message): never
{
    setFlash($message, 'error');
    redirect('index.php');
}

function handleRegister(string $name, string $password): array
{
    $error = registrationError($name, $password);

    if ($error !== null) {
        failAuthentication($error);
    }

    return registerUser($name, $password);
}

function handleLogin(string $name, string $password): array
{
    $user = authenticate($name, $password);

    if ($user === null) {
        failAuthentication('Nome ou senha inválidos.');
    }

    return $user;
}
```

FUNÇÃO	VARIÁVEIS E RECURSOS	O QUE FAZ
`authenticate($name, $password)`	`$user`: usuário encontrado, ou `null`	Duas guard clauses: se o usuário não existe ou a senha não bate, devolve `null`. Senão, devolve o usuário.
`registrationError($name, $password)`	usa `isValidName`, `isValidPassword`, `findUserByName`	Devolve a mensagem do **primeiro** problema do cadastro, ou `null` se está tudo certo.
`failAuthentication($message)`	retorno `never`	Guarda a mensagem de erro e volta para `index.php`.
`handleRegister($name, $password)`	`$error`: mensagem de erro ou `null`	Se houver erro, chama `failAuthentication()` (o script para ali). Senão, cadastra e devolve o usuário.
`handleLogin($name, $password)`	`$user`	Se a autenticação falhar, mostra "Nome ou senha inválidos." e volta. Senão, devolve o usuário.

includes/ranking.php — melhor partida e ranking

```php
<?php

declare(strict_types=1);

function hasPlayed(array $user): bool
{
    return $user['best_time_seconds'] !== null;
}

function isBetterGame(array $user, int $correctAnswers, float $time): bool
{
    if (!hasPlayed($user)) {
        return true;
    }

    if ($correctAnswers !== $user['best_correct_answers']) {
        return $correctAnswers > $user['best_correct_answers'];
    }

    return $time < $user['best_time_seconds'];
}

function saveBestGame(int $userId, int $correctAnswers, float $time): void
{
    $user = findUserById($userId);

    if ($user === null || !isBetterGame($user, $correctAnswers, $time)) {
        return;
    }

    $user['best_correct_answers'] = $correctAnswers;
    $user['best_time_seconds'] = round($time, 2);
    replaceUser($user);
}

function compareRanking(array $first, array $second): int
{
    return [$second['best_correct_answers'], $first['best_time_seconds']]
        <=> [$first['best_correct_answers'], $second['best_time_seconds']];
}

function rankedUsers(): array
{
    $players = array_filter(listUsers(), 'hasPlayed');
    usort($players, 'compareRanking');

    return $players;
}
```

FUNÇÃO	VARIÁVEIS E RECURSOS	O QUE FAZ
`hasPlayed($user)`	`best_time_seconds !== null`	Diz se o usuário já terminou alguma partida.
`isBetterGame($user, $correctAnswers, $time)`	`$correctAnswers`: acertos da partida nova. `$time`: tempo total dela.	1) Nunca jogou: é melhor. 2) Acertos diferentes: vence quem acertou mais. 3) Mesmos acertos: vence o menor tempo.

`saveBestGame($userId, $correctAnswers, $time)`	`$user`: usuário atual. `round($time, 2)`: 2 casas decimais.	Só grava se a partida for melhor que o recorde (*tell, don't ask*).
`compareRanking($first, $second)`	Operador `<=>` (nave espacial) comparando arrays	Compara primeiro os acertos (invertidos, para o maior vir antes) e, se empatar, o tempo (menor antes). Devolve -1, 0 ou 1, como o `usort` espera.
`rankedUsers()`	`$players`: quem já jogou. `array_filter(..., 'hasPlayed')`, `usort(..., 'compareRanking')`	Tira quem nunca jogou e ordena pelo critério do ranking. Os nomes entre aspas são funções passadas como *callback*.

includes/questions.php — perguntas

```php
<?php

declare(strict_types=1);

function listQuestions(): array
{
    return readJson(QUESTIONS_FILE);
}

function findQuestionById(int $id): ?array
{
    return findFirst(listQuestions(), fn (array $question) => $question['id'] === $id);
}

function drawQuestionIds(int $amount): array
{
    $ids = array_column(listQuestions(), 'id');
    shuffle($ids);

    return array_slice($ids, 0, $amount);
}

function isValidOption(array $question, int $option): bool
{
    return array_key_exists($option, $question['options']);
}

function isCorrectAnswer(array $question, int $option): bool
{
    return $question['answer'] === $option;
}

function correctOptionText(array $question): string
{
    return $question['options'][$question['answer']];
}
```

FUNÇÃO	VARIÁVEIS E RECURSOS	O QUE FAZ
`listQuestions()`	`readJson(QUESTIONS_FILE)`	Lê todas as perguntas.

`findQuestionById($id)`	arrow function	Busca uma pergunta pelo id.
`drawQuestionIds($amount)`	`$ids`: todos os ids. `shuffle` (embaralha), `array_slice` (corta)	Embaralha os ids e pega os primeiros `$amount`. Como cada id aparece uma vez, não há repetição.
`isValidOption($question, $option)`	`array_key_exists`	Diz se o índice enviado existe entre as opções (0 a 3).
`isCorrectAnswer($question, $option)`	`===`	Compara o índice enviado com o `answer`.
`correctOptionText($question)`	`$question['options'][$question['answer']]`	Texto da opção correta, usado em "Errou! A resposta era ...".

includes/game.php — estado da partida

A partida fica guardada na sessão, em `$_SESSION['game']`:

```
'question_ids'        => [12, 4, 37, ...]   ids sorteados
'current_index'       => 0                  posição da pergunta atual (0 a 9)
'correct_answers'     => 0                  acertos até agora
'question_started_at' => 1730390400.52      quando a pergunta apareceu (ou null)
'total_time_seconds'  => 0.0                soma do tempo das respostas
```

```php
<?php

declare(strict_types=1);

function gameValue(string $key): mixed
{
    return $_SESSION['game'][$key];
}

function setGameValue(string $key, mixed $value): void
{
    $_SESSION['game'][$key] = $value;
}

function startGame(): void
{
    $_SESSION['game'] = [
        'question_ids' => drawQuestionIds(QUESTIONS_PER_GAME),
        'current_index' => 0,
        'correct_answers' => 0,
        'question_started_at' => null,
        'total_time_seconds' => 0.0,
    ];
}

function isGameInProgress(): bool
{
    return isset($_SESSION['game']);
}
```

```
function totalQuestions(): int
{
    return count(gameValue('question_ids'));
}

function currentQuestionNumber(): int
{
    return gameValue('current_index') + 1;
}

function currentQuestionId(): int
{
    $questionIds = gameValue('question_ids');

    return $questionIds[gameValue('current_index')];
}

function isGameOver(): bool
{
    return gameValue('current_index') >= totalQuestions();
}

function correctAnswersCount(): int
{
    return gameValue('correct_answers');
}

function registerCorrectAnswer(): void
{
    setGameValue('correct_answers', correctAnswersCount() + 1);
}

function advanceQuestion(): void
{
    setGameValue('current_index', gameValue('current_index') + 1);
}
```

FUNÇÃO	VARIÁVEIS E RECURSOS	O QUE FAZ
`gameValue($key)`	`$key`: nome do campo. Retorno `mixed` (qualquer tipo).	Único ponto de leitura de `$_SESSION['game']` (regra "um ponto por linha").
`setGameValue($key, $value)`	`$value`: novo valor	Único ponto de escrita em `$_SESSION['game']`.
`startGame()`	`drawQuestionIds(QUESTIONS_PER_GAME)`	Cria a partida: sorteia 10 perguntas e zera índice, acertos e tempo.
`isGameInProgress()`	`isset`	Diz se existe partida na sessão.
`totalQuestions()`	`count`	Quantas perguntas a partida tem.
`currentQuestionNumber()`		Índice + 1, para exibir "Pergunta 3 de 10".
`currentQuestionId()`	`$questionIds`: lista sorteada	Id da pergunta na posição atual.

`isGameOver()`	`>=`	Verdadeiro quando o índice passou da última pergunta.
`correctAnswersCount()`		Acertos até agora.
`registerCorrectAnswer()`		Soma 1 acerto.
`advanceQuestion()`		Avança o índice para a próxima pergunta.

`includes/timer.php` — cronômetro

O tempo é medido **no servidor**, para o jogador não conseguir alterá-lo.

```
<?php

declare(strict_types=1);

function startQuestionTimer(): void
{
    if (isQuestionTimerRunning()) {
        return;
    }

    setGameValue('question_started_at', microtime(true));
}

function isQuestionTimerRunning(): bool
{
    return gameValue('question_started_at') !== null;
}

function elapsedQuestionTime(): float
{
    return microtime(true) - gameValue('question_started_at');
}

function stopQuestionTimer(): float
{
    $elapsed = elapsedQuestionTime();
    setGameValue('total_time_seconds', totalGameTime() + $elapsed);
    setGameValue('question_started_at', null);

    return $elapsed;
}

function totalGameTime(): float
{
    return gameValue('total_time_seconds');
}
```

FUNÇÃO	VARIÁVEIS E RECURSOS	O QUE FAZ
`startQuestionTimer()`	`microtime(true)`: segundos desde 1970, com frações (ex.: `1730390400.52`)	Marca o início da pergunta. Se já estiver rodando, não faz nada: por isso o F5 não zera o cronômetro.

`isQuestionTimerRunning()`		Verdadeiro se há um início marcado.
`elapsedQuestionTime()`	agora − início	Segundos desde que a pergunta apareceu.
`stopQuestionTimer()`	`$elapsed`: tempo desta pergunta	Soma o tempo ao total da partida, limpa o início e devolve o tempo da pergunta.
`totalGameTime()`		Tempo total acumulado.

includes/result.php — feedback e fim da partida

```php
<?php

declare(strict_types=1);

function answerFeedback(array $question, bool $isCorrect, float $elapsed): string
{
    if ($isCorrect) {
        return 'Acertou! (' . formatTime($elapsed) . ')';
    }

    return 'Errou! A resposta era "' . correctOptionText($question) . '".';
}

function buildGameResult(array $user): array
{
    $correctAnswers = correctAnswersCount();
    $time = totalGameTime();

    return [
        'correct_answers' => $correctAnswers,
        'total_questions' => totalQuestions(),
        'time_seconds' => $time,
        'is_new_record' => isBetterGame($user, $correctAnswers, $time),
    ];
}

function finishGame(int $userId): void
{
    // O resultado é montado antes de salvar, senão a partida nunca seria "novo recorde".
    $_SESSION['last_result'] = buildGameResult(findUserById($userId));
    saveBestGame($userId, correctAnswersCount(), totalGameTime());
    unset($_SESSION['game']);
}

function lastGameResult(): ?array
{
    return $_SESSION['last_result'] ?? null;
}
```

FUNÇÃO	VARIÁVEIS E RECURSOS	O QUE FAZ
`answerFeedback($question, $isCorrect, $elapsed)`	`$isCorrect`: acertou ou não. `$elapsed`: tempo da resposta.	"Acertou! (4,2s)" ou "Errou! A resposta era "X"."

`buildGameResult($user)`	`$correctAnswers`, `$time`	Monta o resumo: acertos, total de perguntas, tempo e se é novo recorde (`isBetterGame`).
`finishGame($userId)`	`$_SESSION['last_result']`, `unset($_SESSION['game'])`	1) Guarda o resumo. 2) Salva o recorde, se for melhor. 3) Apaga a partida. O resumo é montado **antes** de salvar; depois disso a partida nunca seria "melhor que ela mesma".
`lastGameResult()`	`?? null`	Resumo da última partida, usado pela tela de resultado.

4. Templates — templates/

Pedaços de HTML repetidos em todas as páginas. As páginas definem algumas variáveis **antes** de incluir o template:

VARIÁVEL	OBRIGATÓRIA	PARA QUÊ
`$pageTitle`	sim	Texto da aba do navegador.
`$pageStyle`	não	CSS específico da página.
`$pageScript`	não	JS específico da página.

templates/header.php

```
<?php $flash = pullFlash(); ?>
<!DOCTYPE html>
<html lang="pt-br">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title><?= escape($pageTitle) ?> | Quiz Muito Louco!</title>
  <link rel="stylesheet" href="<?= url('assets/global.css') ?>">
  <?php if (isset($pageStyle)): ?>
    <link rel="stylesheet" href="<?= url($pageStyle) ?>">
  <?php endif; ?>
</head>

<body>
  <header class="topbar">
    <a class="brand" href="<?= url('index.php') ?>">Quiz Muito Louco!</a>
    <?php if (loggedUserId() !== null): ?>
      <nav class="nav">
        <a href="<?= url('pages/game/game.php') ?>">Jogar</a>
        <a href="<?= url('pages/ranking/ranking.php') ?>">Ranking</a>
        <a href="<?= url('logout.php') ?>">Sair</a>
      </nav>
    <?php endif; ?>
  </header>

  <?php if ($flash !== null): ?>
    <p class="flash flash-<?= escape($flash['type']) ?>"><?= escape($flash['message']) ?></p>
  <?php endif; ?>
```

- `$flash = pullFlash()`: pega a mensagem pendente (se houver) e a remove da sessão.
- `<?= ... ?>`: forma curta de `<?php echo ... ?>`.
- `if (...): ... endif;`: sintaxe alternativa do `if`, mais legível no meio do HTML.
- `isset($pageStyle)`: só inclui o CSS da página se ela tiver definido um.
- O menu (Jogar, Ranking, Sair) só aparece quando `loggedUserId()` não é `null`.
- A mensagem flash recebe a classe `flash-success` ou `flash-error`, que define a cor.

templates/footer.php

```
  <script src="<?= url('assets/script.js') ?>"></script>
  <?php if (isset($pageScript)): ?>
    <script src="<?= url($pageScript) ?>"></script>
  <?php endif; ?>
</body>

</html>
```

Inclui o `script.js` global e, se a página definiu `$pageScript`, o script dela. Fecha `</body>` e `</html>`.

5. Páginas, na ordem do jogador

Toda página segue o mesmo formato: carrega o `boot.php`, faz as verificações no topo, prepara as variáveis e só então inclui o HTML.

`index.php` — tela inicial

```
<?php

declare(strict_types=1);

require_once __DIR__ . '/includes/boot.php';

if (loggedUserId() !== null) {
    redirect('pages/game/game.php');
}

$pageTitle = 'Entrar';

require ROOT_PATH . '/templates/header.php';
?>
<main class="card">
  <h1>Quiz Muito Louco!</h1>
  <p class="subtitle">
    Responda <?= QUESTIONS_PER_GAME ?> perguntas o mais rápido que conseguir.
    Quem acertar mais fica no topo do ranking; no empate, vence o mais rápido.
  </p>

  <form class="form" method="post" action="<?= url('authenticate.php') ?>">
    <label for="name">Nome</label>
    <input type="text" id="name" name="name" required autocomplete="username"
      minlength="<?= NAME_MIN_LENGTH ?>" maxlength="<?= NAME_MAX_LENGTH ?>">

    <label for="password">Senha</label>
    <input type="password" id="password" name="password" required autocomplete="current-password"
      minlength="<?= PASSWORD_MIN_LENGTH ?>">

    <div class="actions">
      <button class="button" type="submit" name="action" value="login">Entrar</button>
      <button class="button button-secondary" type="submit" name="action"
value="register">Cadastrar</button>
    </div>
  </form>
</main>
<?php require ROOT_PATH . '/templates/footer.php'; ?>
```

TRECHO	O QUE FAZ
`loggedUserId() !== null`	Quem já está logado vai direto para o jogo.
`$pageTitle`	Título da aba: "Entrar".
Formulário	Envia `name`, `password` e `action` para `authenticate.php`. Os dois botões têm `name="action"`: o clicado envia `login` ou `register`.

`minlength`/`maxlength`	Validação no navegador usando as mesmas constantes do PHP. O servidor valida de novo.

authenticate.php — processa login e cadastro

```
<?php

declare(strict_types=1);

require_once __DIR__ . '/includes/boot.php';

if (!isPostRequest()) {
    redirect('index.php');
}

$name = trim((string) filter_input(INPUT_POST, 'name'));
$password = (string) filter_input(INPUT_POST, 'password');
$isRegistration = filter_input(INPUT_POST, 'action') === 'register';

$user = $isRegistration ? handleRegister($name, $password) : handleLogin($name, $password);

startUserSession($user);
setFlash('Bem-vindo, ' . $user['name'] . '!');
redirect('pages/game/game.php');
```

VARIÁVEL / RECURSO	O QUE FAZ
`filter_input(INPUT_POST, 'name')`	Lê um campo do formulário com segurança. Devolve `null` se o campo não veio.
`(string)`	Converte `null` em `''`, garantindo o tipo `string`.
`$name`	Nome digitado, sem espaços nas pontas (`trim`).
`$password`	Senha digitada.
`$isRegistration`	`true` se o botão clicado foi "Cadastrar".
`$user`	Usuário devolvido por `handleRegister` ou `handleLogin` (escolhido pelo ternário). Em caso de erro essas funções já redirecionam, então daqui em diante o login deu certo.

pages/game/game.php — tela da pergunta

```
<?php

declare(strict_types=1);

require_once __DIR__ . '/../../includes/boot.php';

requireLogin();

if (!isGameInProgress()) {
```

```
    startGame();
}

startQuestionTimer();

$question = findQuestionById(currentQuestionId());
$elapsedSeconds = round(elapsedQuestionTime(), 1);
$questionNumber = currentQuestionNumber();
$totalQuestions = totalQuestions();

$pageTitle = 'Pergunta ' . $questionNumber;
$pageStyle = 'pages/game/css/game.css';
$pageScript = 'pages/game/js/game.js';

require ROOT_PATH . '/templates/header.php';
?>
<main class="card">
  <div class="game-status">
    <span>Pergunta <?= $questionNumber ?> de <?= $totalQuestions ?></span>
    <span class="timer" data-elapsed="<?= $elapsedSeconds ?>"><?= formatTime($elapsedSeconds) ?></span>
  </div>
  <progress class="progress" value="<?= $questionNumber - 1 ?>" max="<?= $totalQuestions ?>"></progress>

  <h1 class="question"><?= escape($question['question']) ?></h1>

  <form class="answer-form" method="post" action="<?= url('pages/game/answer.php') ?>">
    <input type="hidden" name="question_id" value="<?= $question['id'] ?>">

    <?php foreach ($question['options'] as $index => $option): ?>
      <label class="option">
        <input type="radio" name="option" value="<?= $index ?>" required>
        <span><?= escape($option) ?></span>
      </label>
    <?php endforeach; ?>

    <button class="button" type="submit">Responder</button>
  </form>
</main>
<?php require ROOT_PATH . '/templates/footer.php'; ?>
```

VARIÁVEL / TRECHO	O QUE FAZ
`requireLogin()`	Só usuários logados jogam.
`startGame()`	Cria a partida, caso ainda não exista.
`startQuestionTimer()`	Começa a contar o tempo da pergunta (se já começou, mantém).
`$question`	A pergunta atual (enunciado, opções, resposta).
`$elapsedSeconds`	Tempo já decorrido, com 1 casa. Vai no atributo `data-elapsed` para o cronômetro da tela continuar do ponto certo após um F5.
`$questionNumber`, `$totalQuestions`	Para "Pergunta 3 de 10" e para a barra de progresso.
`foreach (... as $index => $option)`	Cria um `radio` por opção; o `value` é o índice (0 a 3), e não o texto.

`question_id` (campo oculto)	Permite ao `answer.php` conferir se a resposta é da pergunta atual.

pages/game/answer.php — processa a resposta

```
<?php

declare(strict_types=1);

require_once __DIR__ . '/../../includes/boot.php';

requireLogin();

if (!isPostRequest() || !isGameInProgress() || !isQuestionTimerRunning()) {
    redirect('pages/game/game.php');
}

// Descarta envios de uma aba antiga, que responderiam a pergunta errada.
if (filter_input(INPUT_POST, 'question_id', FILTER_VALIDATE_INT) !== currentQuestionId()) {
    redirect('pages/game/game.php');
}

$question = findQuestionById(currentQuestionId());
$option = filter_input(INPUT_POST, 'option', FILTER_VALIDATE_INT);

if (!is_int($option) || !isValidOption($question, $option)) {
    setFlash('Escolha uma das opções.', 'error');
    redirect('pages/game/game.php');
}

$elapsed = stopQuestionTimer();
$isCorrect = isCorrectAnswer($question, $option);

if ($isCorrect) {
    registerCorrectAnswer();
}

setFlash(answerFeedback($question, $isCorrect, $elapsed), $isCorrect ? 'success' : 'error');
advanceQuestion();

if (isGameOver()) {
    finishGame(loggedUserId());
    redirect('pages/game/result.php');
}

redirect('pages/game/game.php');
```

VARIÁVEL / TRECHO	O QUE FAZ
1º `if`	Sem POST, sem partida ou com o cronômetro parado (envio duplicado): volta para o jogo.
2º `if`	A resposta veio de uma aba antiga (outra pergunta): descarta.
`FILTER_VALIDATE_INT`	Converte o campo em inteiro; se não for número, devolve `false`.
`$question`	Pergunta atual.

`$option`	Índice escolhido. `is_int` + `isValidOption` barram valores como `9` ou `"abc"`.
`$elapsed`	Tempo da resposta (`stopQuestionTimer`).
`$isCorrect`	Se acertou. Define o texto e a cor (`success`/`error`) da mensagem.
`isGameOver()`	Na última pergunta, `finishGame()` encerra a partida e vai para o resultado.

pages/game/result.php — resultado

```
<?php

declare(strict_types=1);

require_once __DIR__ . '/../../includes/boot.php';

requireLogin();

$result = lastGameResult();

if ($result === null) {
    redirect('pages/game/game.php');
}

$pageTitle = 'Resultado';
$pageStyle = 'pages/game/css/game.css';

require ROOT_PATH . '/templates/header.php';
?>
<main class="card result">
  <h1>Fim de jogo!</h1>

  <?php if ($result['is_new_record']): ?>
    <p class="badge">Novo recorde!</p>
  <?php endif; ?>

  <p class="score">
    <strong><?= $result['correct_answers'] ?></strong> de <?= $result['total_questions'] ?> acertos
  </p>
  <p class="total-time">Tempo total: <?= formatTime($result['time_seconds']) ?></p>

  <div class="actions">
    <a class="button" href="<?= url('pages/game/game.php') ?>">Jogar novamente</a>
    <a class="button button-secondary" href="<?= url('pages/ranking/ranking.php') ?>">Ver ranking</a>
  </div>
</main>
<?php require ROOT_PATH . '/templates/footer.php'; ?>
```

`$result` é o resumo gravado por `finishGame()`, com `correct_answers`, `total_questions`, `time_seconds` e `is_new_record`. A página **só exibe**; como nada é gravado aqui, o F5 é seguro.

pages/ranking/ranking.php — ranking

```
<?php
```

```php
declare(strict_types=1);

require_once __DIR__ . '/../../includes/boot.php';

requireLogin();

$rankedUsers = rankedUsers();
$loggedUserId = loggedUserId();

$pageTitle = 'Ranking';
$pageStyle = 'pages/ranking/css/ranking.css';

require ROOT_PATH . '/templates/header.php';
?>
<main class="card card-wide">
  <h1>Ranking</h1>
  <p class="subtitle">Mais acertos primeiro; no empate, vence o menor tempo.</p>

  <?php if ($rankedUsers === []): ?>
    <p class="empty">Ninguém jogou ainda. Seja o primeiro!</p>
  <?php endif; ?>

  <?php if ($rankedUsers !== []): ?>
    <table class="ranking">
      <thead>
        <tr>
          <th>#</th>
          <th>Jogador</th>
          <th>Acertos</th>
          <th>Tempo</th>
        </tr>
      </thead>
      <tbody>
        <?php foreach ($rankedUsers as $position => $user): ?>
          <tr class="<?= $user['id'] === $loggedUserId ? 'is-current-user' : '' ?>">
            <td><?= $position + 1 ?>º</td>
            <td><?= escape($user['name']) ?></td>
            <td><?= $user['best_correct_answers'] ?>/<?= QUESTIONS_PER_GAME ?></td>
            <td><?= formatTime($user['best_time_seconds']) ?></td>
          </tr>
        <?php endforeach; ?>
      </tbody>
    </table>
  <?php endif; ?>
</main>
<?php require ROOT_PATH . '/templates/footer.php'; ?>
```

VARIÁVEL	O QUE FAZ
`$rankedUsers`	Jogadores que já jogaram, já ordenados.
`$loggedUserId`	Para destacar a linha do usuário logado (classe `is-current-user`).
`$position`	Índice da lista (0, 1, 2...); exibido como `$position + 1` (1º, 2º...).
`$user`	Jogador da linha. O nome passa por `escape()`; a senha nunca é exibida.

logout.php — sair

```php
<?php

declare(strict_types=1);

require_once __DIR__ . '/includes/boot.php';

endUserSession();
redirect('index.php');
```

Destrói a sessão (`endUserSession`) e volta para a tela inicial.

6. Estilos e scripts

ARQUIVO	CONTEÚDO
`assets/global.css`	Variáveis de cor em `:root`, fundo em degradê, cartão central, menu, formulários, botões e mensagens flash.
`pages/game/css/game.css`	Progresso, cronômetro, opções clicáveis (a marcada é destacada só com CSS: `.option:has(input:checked)`) e placar do resultado.
`pages/ranking/css/ranking.css`	Tabela, cores do 1º, 2º e 3º lugares e destaque do usuário logado.

assets/script.js

```
const FLASH_DURATION_MS = 4000;

const flashMessage = document.querySelector('.flash');

if (flashMessage) {
  setTimeout(() => flashMessage.classList.add('is-hidden'), FLASH_DURATION_MS);
}
```

`FLASH_DURATION_MS` (4000 ms) define quanto tempo a mensagem fica visível; depois ela recebe a classe `is-hidden` e some com transição.

pages/game/js/game.js

O jogo funciona sem JavaScript; o script só melhora a experiência.

```
const TIMER_INTERVAL_MS = 100;

const answerForm = document.querySelector('.answer-form');
const submitButton = answerForm.querySelector('button[type="submit"]');
const timer = document.querySelector('.timer');
const startedAt = Date.now() - Number(timer.dataset.elapsed) * 1000;

function formatTime(seconds) {
  const minutes = Math.floor(seconds / 60);
  const remainingSeconds = (seconds - minutes * 60).toFixed(1).replace('.', ',');

  if (minutes === 0) {
    return `${remainingSeconds}s`;
  }

  return `${minutes}min ${remainingSeconds}s`;
}

function updateTimer() {
  timer.textContent = formatTime((Date.now() - startedAt) / 1000);
}

function hasSelectedOption() {
  return answerForm.querySelector('input[name="option"]:checked') !== null;
```

```
}

function updateSubmitButton() {
  submitButton.disabled = !hasSelectedOption();
}

updateSubmitButton();
setInterval(updateTimer, TIMER_INTERVAL_MS);
answerForm.addEventListener('change', updateSubmitButton);
answerForm.addEventListener('submit', () => {
  submitButton.disabled = true;
});
```

ITEM	TIPO	O QUE FAZ
`TIMER_INTERVAL_MS`	constante	Intervalo de atualização do cronômetro (100 ms).
`answerForm`, `submitButton`, `timer`	elementos	Formulário, botão "Responder" e cronômetro da página.
`startedAt`	número	Momento (em ms) em que a pergunta começou: agora − `data-elapsed`.
`formatTime(seconds)`	função	Mesmo formato do PHP: "1min 23,4s".
`updateTimer()`	função	Atualiza o texto do cronômetro. Só visual: o tempo que vale é o do servidor.
`hasSelectedOption()`	função	Diz se alguma opção está marcada.
`updateSubmitButton()`	função	Habilita "Responder" só com uma opção marcada.
`submit` (evento)	evento	Desabilita o botão ao enviar, evitando duplo clique.

7. Glossário de funções e recursos do PHP

Funções nativas usadas

FUNÇÃO	O QUE FAZ
`session_start()`	Inicia ou retoma a sessão.
`session_regenerate_id(true)`	Troca o id da sessão e apaga o antigo.
`session_destroy()`	Destrói a sessão.
`require_once` / `require`	Inclui outro arquivo PHP (`_once`: só uma vez).
`file_get_contents($path)`	Lê o arquivo inteiro e devolve uma string (ou `false`).
`file_put_contents($path, $text, LOCK_EX)`	Grava texto no arquivo, com trava exclusiva.
`json_decode($text, true)`	Texto JSON → array.
`json_encode($array, $flags)`	Array → texto JSON.
`trim($text)`	Remove espaços do início e do fim.
`htmlspecialchars(...)`	Escapa caracteres especiais do HTML.
`header('Location: ...')`	Envia ao navegador a ordem de ir para outra URL.
`exit`	Encerra o script imediatamente.
`filter_input(INPUT_POST, ...)`	Lê (e opcionalmente valida) um campo enviado por POST.
`array_filter($list, $callback)`	Mantém só os itens em que o callback devolve `true`.
`array_map($callback, $list)`	Cria uma lista nova aplicando o callback a cada item.
`array_values($list)`	Renumera os índices a partir de 0.
`array_column($list, 'id')`	Extrai uma coluna (ex.: todos os ids).
`array_slice($list, 0, $n)`	Pega os `$n` primeiros itens.
`array_key_exists($key, $list)`	Diz se a posição/chave existe.
`shuffle($list)`	Embaralha a lista.
`usort($list, $callback)`	Ordena a lista usando a função de comparação.
`count($list)`	Quantidade de itens.
`max($list)`	Maior valor.

`mb_strtolower` / `mb_strlen`	Minúsculas / tamanho, com suporte a acentos.
`microtime(true)`	Hora atual em segundos, com frações.
`round($n, 2)`	Arredonda para 2 casas.
`intdiv($a, $b)`	Divisão inteira.
`number_format($n, 1, ',', '')`	Formata com 1 casa e vírgula decimal.
`isset` / `unset`	Verifica se existe / remove uma variável ou chave.
`is_int($value)`	Diz se o valor é inteiro.

Recursos da linguagem

RECURSO	SIGNIFICADO
`declare(strict_types=1)`	Tipagem estrita: tipos errados geram erro.
`const NOME = valor`	Constante: valor fixo que não muda.
`__DIR__`	Pasta do arquivo atual.
`$_SESSION`	Dados do visitante guardados entre uma página e outra.
`$_SERVER`	Informações da requisição (ex.: método GET/POST).
`?array`, `?int`, `?float`	Tipo "anulável": o valor ou `null`.
`mixed`	Qualquer tipo.
`never`	A função nunca retorna (sempre termina com `exit`).
`callable`	Uma função passada como valor (callback).
`fn ($x) => ...`	Arrow function: função curta e sem nome.
`$a ?? $b`	Usa `$a` se existir e não for `null`; senão, `$b`.
`$c ? $a : $b`	Ternário: `$a` se `$c` for verdadeiro; senão, `$b`.
`===` / `!==`	Igual / diferente, comparando valor e tipo.
`<=>`	Nave espacial: devolve -1, 0 ou 1 (menor, igual, maior).
`...$lista`	Espalha os itens da lista dentro de outra.
`.`	Concatena (junta) textos.
`<?= ... ?>`	Imprime um valor no HTML.